BRK (00)	Cycles: 7	Size: 2 *
	Implied	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	** Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand.
2.1		Inc. **PC**.
2.2	*** Write to (**S**+0) | $0100	Push **PC**.H onto stack.
3.1		
3.2	*** Write to (**S**-1) | $0100	Push **PC**.L onto stack.
4.1		Use Interrupt flags to decide which vector to use. RESET = $FFFC. NMI = $FFFA. IRQ/BRK = $FFFE.
4.2	*** Write to (**S**-2) | $0100	Push **P** onto stack with **B** flag if software BRK.
5.1		Dec. **S** by 3.
5.2	Read Vector	Store to Address.L.
6.1		Set **I**, unset **D**.
6.2	Read Vector + 1	Store to Address.H.
7.1		Enable writes to PC (disabled by NMI/IRQ). Copy Address to PC. Set PB to 0.
7.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

* Concerning the BRK instruction, you should also note that although its second byte is basically a “don’t care” byte – that is, it can have any value - the BRK (and COP instruction as well) is a two-byte instruction, the second byte sometimes is used as a signature byte to determine the nature of the BRK being executed. When an RTI instruction is executed, control always returns to the second byte past the BRK opcode. — assembly-programming-manual-for-w65c816.pdf

** If NMI, IRQ, or RESET, OpCode is forced to 0 and writes to PC are disabled.

*** If handling a RESET, the writes turn into reads. The databus is populated but ignored.

ORA (01)	Cycles: 6	Size: 2
	Direct Indexed Indirect (d,x) (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Pointer.
2.1		Inc. **PC**. Set Pointer to Pointer + **X** + **D**. Address.L = Pointer.L. Address.H = **D**.H.
2.2		If **D**.L is 0, skip the next cycle.
3.1		Address.H = Pointer.H (fixes high byte of address).
3.2		
4.1		
4.2	Read Address	Store as Pointer.
5.1		
5.2	Read Address + 1	Store as Pointer.H.
6.1		
6.2	Read Pointer:DB	Store as Operand.
7.1		Perform **A** |= Operand. Set **N** based off (**A**.L & $80) and **Z** based off **A**.L.
7.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

COP (02)	Cycles: 7	Size: 2
	Implied	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand.
2.1		Inc. **PC**.
2.2	Write to (**S**+0) | $0100	Push **PC**.H onto stack.
3.1		
3.2	Write to (**S**-1) | $0100	Push **PC**.L onto stack.
4.1		Set Vector to $FFF4.
4.2	Write to (**S**-2) | $0100	Push **P** onto stack.
5.1		Dec. **S** by 3.
5.2	Read Vector	Store to Address.L.
6.1		Set **I**, unset **D**.
6.2	Read Vector + 1	Store to Address.H.
7.1		Copy Address to **PC**. Set **PB** to 0.
7.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

ORA (03)	Cycles: 4	Size: 2
	Stack Relative (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Pointer.
2.1		Inc. **PC**. Set Address to Pointer + **S**.L | $100.
2.2		
3.1		
3.2	Read Address	Store as Operand.
4.1		Perform **A** |= Operand. Set **N** based off (**A**.L & $80) and **Z** based off **A**.L.
4.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

TSB (04)	Cycles: 5	Size: 2
	Direct (Read/Modify/Write)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand. If **D**.L is 0, skip the next cycle.
2.1		Inc. **PC**.
2.2		
3.1		Inc. **PC** if previous cycle was skipped. Set Address to Operand + **D**.
3.2	Read Address	Store as Operand.
4.1		
4.2	Write to Address	Write Operand.L.
5.1		Set **Z** based off (**A**.L & Operand.L). Perform Operand |= **A**.
5.2	Write to Address	Write Operand.L.
6.1		
6.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

ORA (05)	Cycles: 3	Size: 2
	Direct (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand. If **D**.L is 0, skip the next cycle.
2.1		Inc. **PC**.
2.2		
3.1		Inc. **PC** if previous cycle was skipped. Set Address to Operand + **D**.
3.2	Read Address	Store as Operand.
4.1		Perform **A** |= Operand. Set **N** based off (**A**.L & $80) and **Z** based off **A**.L.
4.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

ASL (06)	Cycles: 5	Size: 2
	Direct (Read/Modify/Write)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand. If **D**.L is 0, skip the next cycle.
2.1		Inc. **PC**.
2.2		
3.1		Inc. **PC** if previous cycle was skipped. Set Address to Operand + **D**.
3.2	Read Address	Store as Operand.
4.1		
4.2	Write to Address	Write Operand.L.
5.1		Set **C** based off (Operand.L & $80). Perform Operand.L <<= 1. Set **N** based off (Operand.L & $80) and **Z** based off Operand.L.
5.2	Write to Address	Write Operand.L.
6.1		
6.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

ORA (07)	Cycles: 6	Size: 2
	Direct Indirect Long (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand. If **D**.L is 0, skip the next cycle.
2.1		Inc. **PC**.
2.2		
3.1		Inc. **PC** if previous cycle was skipped. Set Address to Operand + **D**.
3.2	Read Address	Store as Pointer.
4.1		
4.2	Read Address + 1	Store as Pointer.H.
5.1		
5.2	Read Address + 2	Store as Bank.
6.1		
6.2	Read Pointer:Bank	Store as Operand.
7.1		Perform **A** |= Operand. Set **N** based off (**A**.L & $80) and **Z** based off **A**.L.
7.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

PHP (08)	Cycles: 3	Size: 1
	Implied	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2		
2.1		Sets Operand.L to **P** with **X** and **M** flags set.
2.2	Write to (**S**+0) | $0100	Push Operand.L onto stack.
3.1		
3.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

ORA (09)	Cycles: 2	Size: 2
	Immediate	
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand.
2.1		Inc. **PC**. Perform **A** |= Operand. Set **N** based off (**A**.L & $80) and **Z** based off **A**.L.
2.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

ASL (0A)	Cycles: 2	Size: 1
	Implied	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Discard.
2.1		Set **C** based off (**A**.L & $80). Perform **A**.L <<= 1. Set **N** based off (**A**.L & $80) and **Z** based off **A**.L.
2.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

PHD (0B)	Cycles: 4	Size: 1
	Implied	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2		
2.1		
2.2	Write to (**S**+0) | $0100	Push **D**.H onto stack.
3.1		
3.2	Write to ((**S** | $0100)-1)	Push **D**.L onto stack.
4.1		
4.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

TSB (0C)	Cycles: 6	Size: 3
	Absolute (Read/Modify/Write)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Address.
2.1		Inc. **PC**.
2.2	Read **PC**:**PB**	Store as Address.L.
3.1		Inc. **PC**.
3.2	Read Address:**DB**	Store as Operand.
4.1		
4.2	Write to Address:**DB**	Write Operand.L.
5.1		Set **Z** based off (**A**.L & Operand.L). Perform Operand |= **A**.
5.2	Write to Address:**DB**	Write Operand.L.
6.1		
6.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

ORA (0D)	Cycles: 4	Size: 3
	Absolute (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Address.
2.1		Inc. **PC**.
2.2	Read **PC**:**PB**	Store as Address.L.
3.1		Inc. **PC**.
3.2	Read Address:**DB**	Store as Operand.
4.1		Perform **A** |= Operand. Set **N** based off (**A**.L & $80) and **Z** based off **A**.L.
4.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

ASL (0E)	Cycles: 6	Size: 3
	Absolute (Read/Modify/Write)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Address.
2.1		Inc. **PC**.
2.2	Read **PC**:**PB**	Store as Address.L.
3.1		Inc. **PC**.
3.2	Read Address:**DB**	Store as Operand.
4.1		
4.2	Write to Address:**DB**	Write Operand.L.
5.1		Set **C** based off (Operand.L & $80). Perform Operand.L <<= 1. Set **N** based off (Operand.L & $80) and **Z** based off Operand.L.
5.2	Write to Address:**DB**	Write Operand.L.
6.1		
6.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

ORA (0F)	Cycles: 5	Size: 4
	Absolute Long (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Address.
2.1		Inc. **PC**.
2.2	Read **PC**:**PB**	Store as Address.L.
3.1		Inc. **PC**.
3.2	Read **PC**:**PB**	Stores as Bank.
4.1		Inc. **PC**.
4.2	Read Address:Bank	Store as Operand.
5.1		Perform **A** |= Operand. Set **N** based off (**A**.L & $80) and **Z** based off **A**.L.
5.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

BPL (10)	Cycles: 2	Size: 2
	Relative	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**. Decide to branch if **N** is 0.
1.2	Read **PC**:**PB**	Store as Operand. Address = i16(i8(Operand.L)) + **PC**, BoundaryCrossed = Address.H != PC.H. If not branching or not BoundaryCrossed, poll interrupts.
2.1		Inc. **PC**. If not branching, end (next half-cycle is 4.2).
2.2		
3.1		Set **PC**.L to Address.L. If not BoundaryCrossed, end (next half-cycle is 4.2).
3.2		Poll interrupts.
4.1		Set **PC**.H to Address.H
4.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

ORA (11)	Cycles: 5	Size: 2
	Direct Indirect Indexed (d),y	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand. If **D**.L is 0, skip the next cycle.
2.1		Inc. **PC**.
2.2		
3.1		Inc. **PC** if previous cycle was skipped. Set Address to Operand + **D**.
3.2	Read Address	Store as Pointer.
4.1		
4.2	Read Address + 1	Store as Pointer.H.
5.1		Perform Orig = Pointer. Tmp = ui32(Pointer + **Y**). Pointer = ui16(Tmp). If Orig.H == Pointer.H, skip 2 half-cycles. Bank = **DB** + (Tmp >> 16).
5.2		
6.1		
6.2	Read Pointer:Bank	Store as Operand.
7.1		Perform **A** |= Operand. Set **N** based off (**A**.L & $80) and **Z** based off **A**.L.
7.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

ORA (12)	Cycles: 5	Size: 2
	Direct Indirect (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand. If **D**.L is 0, skip the next cycle.
2.1		Inc. **PC**.
2.2		
3.1		Inc. **PC** if previous cycle was skipped. Set Address to Operand + **D**.
3.2	Read Address	Store as Pointer.
4.1		
4.2	Read Address + 1	Store as Pointer.H.
5.1		
5.2	Read Pointer:**DB**	Store as Operand.
6.1		Perform **A** |= Operand. Set **N** based off (**A**.L & $80) and Z based off **A**.L.
6.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

ORA (13)	Cycles: 7	Size: 2
	Stack Relative Indirect Indexed	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**.	Store as Pointer.
2.1		Inc. **PC**. Set Address to Pointer + **S**.L | $100.
2.2		
3.1		
3.2	Read Address	Store as Pointer.
4.1		
4.2	Read Address + 1	Store as Pointer.H.
5.1		Perform Tmp = ui32(Pointer + **Y**). Pointer = ui16(Tmp). Bank = **DB** + (Tmp >> 16).
5.2		
6.1		
6.2	Read Pointer:Bank	Store as Operand.
7.1		Perform **A** |= Operand. Set **N** based off (**A**.L & $80) and **Z** based off **A**.L.
7.2	Read **PC**:**PB**.	Store as OpCode.
+X.1		

TRB (14)	Cycles: 5	Size: 2
	Direct (Read/Modify/Write)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand. If **D**.L is 0, skip the next cycle.
2.1		Inc. **PC**.
2.2		
3.1		Inc. **PC** if previous cycle was skipped. Set Address to Operand + **D**.
3.2	Read Address	Store as Operand.
4.1		
4.2	Write to Address	Write Operand.L.
5.1		Set **Z** based off (**A**.L & Operand.L). Perform Operand &= ~**A**.
5.2	Write to Address	Write Operand.L.
6.1		
6.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

ORA (15)	Cycles: 4	Size: 2
	Direct, X (Read)	
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand.
2.1		Inc. **PC**.
2.2		
3.1		If **D**.L is 0, Address.L = Operand + **X**.L + **D**.L, Address.H = **D**.H, otherwise Address = Operand + **X** + **D**. If **D**.L is 0, skip 2 half-cycles.
3.2		
4.1		
4.2	Read Address	Store as Operand.
5.1		Perform **A** |= Operand. Set **N** based off (**A**.L & $80) and **Z** based off **A**.L.
5.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

ASL (16)	Cycles: 6	Size: 2
	Direct Indexed Indirect (d,x) (Read/Modify/Write)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand.
2.1		Inc. **PC**.
2.2		
3.1		If **D**.L is 0, Address.L = Operand + **X**.L + **D**.L, Address.H = **D**.H, otherwise Address = Operand + **X** + **D**. If **D**.L is 0, skip 2 half-cycles.
3.2		
4.1		
4.2	Read Address	Store as Operand.
5.1		
5.2	Write to Address	Write Operand.L.
6.1		Set **C** based off (Operand.L & $80). Perform Operand.L <<= 1. Set **N** based off (Operand.L & $80) and **Z** based off Operand.L.
6.2	Write to Address	Write Operand.L.
7.1		
7.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

ORA (17)	Cycles: 6	Size: 2
	Direct Indirect Indexed Long [d],y (Read)	
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand. If **D**.L is 0, skip the next cycle.
2.1		Inc. **PC**.
2.2		
3.1		Inc. **PC** if previous cycle was skipped. Set Address to Operand + **D**.
3.2	Read Address	Store as Pointer.
4.1		
4.2	If **D**.l is not 0, read (Address + 1), otherwise read (Address & $FF00) | ui8(Address.L + 1)	Store as Pointer.H.
5.1		
5.2	If **D**.l is not 0, read (Address + 2), otherwise read (Address & $FF00) | ui8(Address.L + 2)	Store as Bank.
6.1		Perform Tmp = ui32(Pointer + **Y**). Pointer = ui16(Tmp). Bank += (Tmp >> 16).
6.2	Read Pointer:Bank	Store as Operand.
7.1		Perform **A** |= Operand. Set **N** based off (**A**.L & $80) and **Z** based off **A**.L.
7.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

CLC (18)	Cycles: 2	Size: 1
	Implied	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Discard.
2.1		Sets the **C** flag to 0.
2.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

ORA (19)	Cycles: 4	Size: 3
	Absolute, Y (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**.	Store as Pointer.
2.1		Inc. **PC**.
2.2	Read **PC**:**PB**.	Store as Pointer.H.
3.1		Inc. **PC**. Perform Orig = Pointer.
		Tmp = ui32(Pointer + **Y**).
		Pointer = ui16(Tmp).
		If Orig.H == Pointer.H, skip 2 half-cycles.
		Bank = **DB** + (Tmp >> 16).
3.2		
4.1		
4.2	Read Pointer:Bank	Store as Operand.
5.1		Perform **A** |= Operand. Set **N** based off (**A**.L & $80) and **Z** based off **A**.L.
5.2	Read **PC**:**PB**.	Store as OpCode.
+X.1		

INC (1A)	Cycles: 2	Size: 1
	Implied	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Discard.
2.1		Perform **A**.L += 1, set **N** based off (**A**.L & $80), set **Z** based off **A**.L.
2.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

TSC (1B)	Cycles: 2	Size: 1
	Implied	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Discard.
2.1		Sets **S** to **A**.
2.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

TRB (1C)	Cycles: 6	Size: 3
	Absolute (Read/Modify/Write)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Address.
2.1		Inc. **PC**.
2.2	Read **PC**:**PB**	Store as Address.L.
3.1		Inc. **PC**.
3.2	Read Address:**DB**	Store as Operand.
4.1		
4.2	Write to Address:**DB**	Write Operand.L.
5.1		Set **Z** based off (**A**.L & Operand.L). Perform Operand &= ~**A**.
5.2	Write to Address:**DB**	Write Operand.L.
6.1		
6.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

ORA (1D)	Cycles: 4	Size: 3
	Absolute, X (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**.	Store as Pointer.
2.1		Inc. **PC**.
2.2	Read **PC**:**PB**.	Store as Pointer.H.
3.1		Inc. **PC**. Perform Orig = Pointer.
		Tmp = ui32(Pointer + **X**).
		Pointer = ui16(Tmp).
		If Orig.H == Pointer.H, skip 2 half-cycles.
		Bank = **DB** + (Tmp >> 16).
3.2		
4.1		
4.2	Read Pointer:Bank	Store as Operand.
5.1		Perform **A** |= Operand. Set **N** based off (**A**.L & $80) and **Z** based off **A**.L.
5.2	Read **PC**:**PB**.	Store as OpCode.
+X.1		

ASL (1E)	Cycles: 7	Size: 3
	Absolute, X (Read/Modiy/Write)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**.	Store as Pointer.
2.1		Inc. **PC**.
2.2	Read **PC**:**PB**.	Store as Pointer.H.
3.1		Inc. **PC**. Perform Tmp = ui32(Pointer + **X**). Pointer = ui16(Tmp). Bank = **DB** + (Tmp >> 16).
3.2		
4.1		
4.2	Read Pointer:Bank	Store as Operand.
5.1		
5.2	Write to Pointer:Bank	Write Operand.L.
6.1		Set **C** based off (Operand.L & $80). Perform Operand.L <<= 1. Set **N** based off (Operand.L & $80) and **Z** based off Operand.L.
6.2	Write to Pointer:Bank	Write Operand.L.
7.1		
7.2	Read **PC**:**PB**.	Store as OpCode.
+X.1		

ORA (1F)	Cycles: 5	Size: 4
	Absolute Long, X (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**.	Store as Pointer.
2.1		Inc. **PC**.
2.2	Read **PC**:**PB**.	Store as Pointer.H.
3.1		Inc. **PC**.
3.2	Read **PC**:**PB**.	Stores as Bank.
4.1		Inc. **PC**. Perform Tmp = ui32(Pointer + **X**). Pointer = ui16(Tmp). Bank += (Tmp >> 16).
4.2	Read Pointer:Bank	Store as Operand.
5.1		Perform **A** |= Operand. Set **N** based off (**A**.L & $80) and **Z** based off **A**.L.
5.2	Read **PC**:**PB**.	Store as OpCode.
+X.1		

JSR (20)	Cycles: 6	Size: 3
	Absolute	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Address.
2.1		Inc. **PC**.
2.2	Read **PC**:**PB**	Store as Address.L.
3.1		Inc. **PC**.
3.2	Read **PC**:**PB**	Discard.
4.1		Dec. **PC**.
4.2	Write to (**S**+0) | $0100	Push **PC**.H onto stack.
5.1		
5.2	Write to (**S**-1) | $0100	Push **PC**.L onto stack.
6.1		Dec. **S** by 2. Perform **PC** = Address.
6.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

AND (21)	Cycles: 6	Size: 2
	Direct Indexed Indirect (d,x) (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**.	Store as Pointer.
2.1		Inc. **PC**. Set Pointer to Pointer + **X** + **D**. Address.L = Pointer.L. Address.H = **D**.H.
2.2		If **D**.L is 0, skip the next cycle.
3.1		Address.H = Pointer.H (fixes high byte of address).
3.2		
4.1		
4.2	Read Address	Store as Pointer.
5.1		
5.2	Read Address + 1	Store as Pointer.H.
6.1		
6.2	Read Pointer:**DB**	Store as Operand.
7.1		Perform **A** &= Operand. Set **N** based off (**A**.L & $80) and **Z** based off **A**.L.
7.2	Read **PC**:**PB**.	Store as OpCode.
+X.1		

JSL (22)	Cycles: 8	Size: 4
	Absolute Long	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Address.
2.1		Inc. **PC**.
2.2	Read **PC**:**PB**	Store as Address.L.
3.1		Inc. **PC**.
3.2	Write to (**S**+0) | $0100	Push **PB** onto stack.
4.1		
4.2	Read (**S**+0) | $0100	Discard.
5.1		
5.2	Read **PC**:**PB**	Stores as Bank.
6.1		
6.2	Write to ((**S** | $0100)-1)	Push **PC**.H onto stack.
7.1		
7.2	Write to ((**S** | $0100)-2)	Push **PC**.L onto stack.
8.1		Dec. **S** by 3. Copy Address to **PC**. Copy Bank to **PB**.
8.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

AND (23)	Cycles: 4	Size: 2
	Stack Relative (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**.	Store as Pointer.
2.1		Inc. **PC**. Set Address to Pointer + (**S**.L | $100).
2.2		
3.1		
3.2	Read Address	Store as Operand.
4.1		Perform **A** &= Operand. Set **N** based off (**A**.L & $80) and **Z** based off **A**.L.
4.2	Read **PC**:**PB**.	Store as OpCode.
+X.1		

BIT (24)	Cycles: 3	Size: 2
	Direct (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand. If **D**.L is 0, skip the next cycle.
2.1		Inc. **PC**.
2.2		
3.1		Inc. **PC** if previous cycle was skipped. Set Address to Operand + **D**.
3.2	Read Address	Store as Operand.
4.1		Set **Z** based off (**A**.L & Operand.L). Set **N** based off (Operand.L & $80). Set **V** based off (Operand.L & $40).
4.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

AND (25)	Cycles: 3	Size: 2
	Direct (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand. If **D**.L is 0, skip the next cycle.
2.1		Inc. **PC**.
2.2		
3.1		Inc. **PC** if previous cycle was skipped. Set Address to Operand + **D**.
3.2	Read Address	Store as Operand.
4.1		Perform **A** &= Operand. Set **N** based off (**A**.L & $80) and **Z** based off **A**.L.
4.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

ROL (26)	Cycles: 5	Size: 2
	Direct (Read/Modify/Write)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand. If **D**.L is 0, skip the next cycle.
2.1		Inc. **PC**.
2.2		
3.1		Inc. **PC** if previous cycle was skipped. Set Address to Operand + **D**.
3.2	Read Address	Store as Operand.
4.1		
4.2	Write to Address	Write Operand.L.
5.1		Set **C** based off (Operand.L & $80). Perform Operand.L = (Operand.L << 1) | **C**. Set **N** based off (Operand.L & $80) and **Z** based off Operand.L.
5.2	Write to Address	Write Operand.L.
6.1		
6.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

AND (27)	Cycles: 6	Size: 2
	Direct Indirect Long (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand. If **D**.L is 0, skip the next cycle.
2.1		Inc. **PC**.
2.2		
3.1		Inc. **PC** if previous cycle was skipped. Set Address to Operand + **D**.
3.2	Read Address	Store as Pointer.
4.1		
4.2	Read Address + 1	Store as Pointer.H.
5.1		
5.2	Read Address + 2	Store as Bank.
6.1		
6.2	Read Pointer:Bank	Store as Operand.
7.1		Perform **A** &= Operand. Set **N** based off (**A**.L & $80) and **Z** based off **A**.L.
7.2	Read **PC**:**PB**	Store as OpCode.
+X.1		